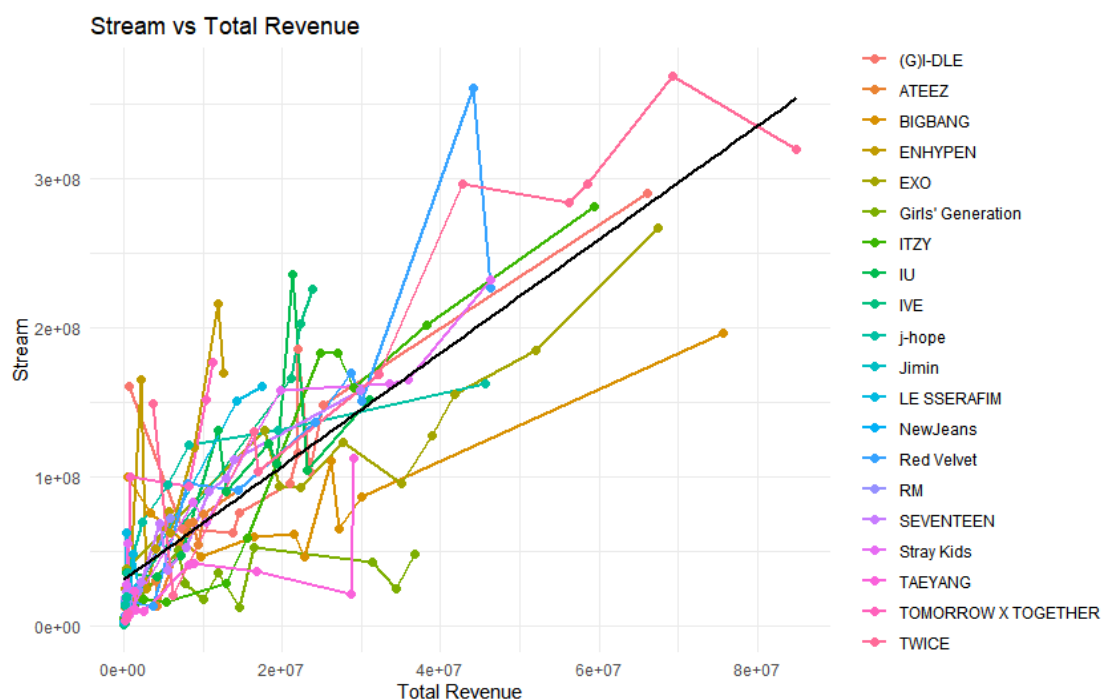


Data vis practice7 -team2

Task1: Based on the feedback received from Practice 6, improve your visualization results and analysis. Include an explanation in the report on how the improvements were made.



Last Feedback

1. 너무 많은 색상과 선들로 인해 보기가 어려움
2. 산점도로 진행하면 더 좋을 것으로 예상

각 아티스트별로 최대 10곡의 노래가 있다보니 위와 같은 피드백이 발생하였다.

따라서 각 아티스트의 모든 곡들의 total_revenue 값을 합하고 그 외의 음악적요소들은 평균값을 계산하여 각 아티스트별로 정보를 요약하여 생성하였다.

이후, 한국 아티스트와 다른 전 세계 아티스트를 산점도로 표시하였고, 각 아티스트별로 서로 다른 색상을 사용할 경우 눈에 띄지 않는 문제를 해결하기 위해, 각 group별로 색상을 정

해, Korean-artist와 global-artist 두 개의 그룹 내에서는 동일한 색상으로 표시하였다.
너무 높은 revenue를 가진 아티스트를 이상치로 처리하여 전 세계적인 추세를 확인하였다.

Task2: Implement the improved visualization results and analysis from step 1 into a Shiny app.

```
# 필요한 라이브러리 로드
library(shiny)
library(ggplot2)
library(dplyr)
library(rsconnect)
library(plotly)

# 음악 데이터를 로드 (Spotify와 YouTube 데이터를 포함한 CSV 파일)
music_data <- read.csv("Spotify_Youtube.csv")

# YouTube 수익 열 추가 (조회수에 따른 수익 계산)
music_data <- music_data %>%
  mutate(
    YouTube_Revenue = ifelse(Views <= 10000, Views * 0.06,
    Views * 0.12)
  )

# Spotify 스트리밍 수익 열 추가 (스트리밍 횟수에 따른 수익 계산)
music_data <- music_data %>%
  mutate(
    Spotify_Revenue = Stream * (0.04 / 10)
  )

# YouTube와 Spotify 수익을 합친 총 수익 열 추가
music_data <- music_data %>%
  mutate(
    Total_Revenue = YouTube_Revenue + Spotify_Revenue
  )

# 각 아티스트별로 요약된 데이터를 담은 새로운 데이터프레임 생성
revenue_data <- music_data %>%
  group_by(Artist) %>%
```

```

summarise(
  Sum_Total_Revenue = sum(Total_Revenue, na.rm = TRUE),
  Avg_Danceability = mean(Danceability, na.rm = TRUE),
  Avg_Energy = mean(Energy, na.rm = TRUE),
  Avg_Loudness = mean(Loudness, na.rm = TRUE),
  Avg_Speechiness = mean(Speechiness, na.rm = TRUE),
  Avg_Acousticness = mean(Acousticness, na.rm = TRUE),
  Avg_Instrumentalness = mean(Instrumentalness, na.rm = T
RUE),
  Avg_Liveness = mean(Liveness, na.rm = TRUE),
  Avg_Valence = mean(Valence, na.rm = TRUE),
  Avg_Tempo = mean(Tempo, na.rm = TRUE),
  Avg_Duration_ms = mean(Duration_ms, na.rm = TRUE),
  Avg_Views = mean(Views, na.rm = TRUE),
  Avg_Likes = mean(Likes, na.rm = TRUE),
  Avg_Comments = mean(Comments, na.rm = TRUE),
  Avg_Stream = mean(Stream, na.rm = TRUE)
)

```

한국 아티스트 목록 정의

```

korean_artists <- c("BTS", "EXO", "BLACKPINK", "TWICE", "St
ray Kids",
                    "Red Velvet", "NCT", "SEVENTEEN", "BIGB
ANG",
                    "SHINee", "(G)I-DLE", "ATEEZ", "TOMORRO
W X TOGETHER",
                    "ENHYPEN", "IVE", "LE SSERAFIM", "NewJe
ans",
                    "TAEYANG", "RM", "Jimin", "j-hope", "I
U",
                    "Girls' Generation", "JAY PARK", "ITZ
Y")

```

한국 아티스트 데이터를 필터링하여 korean_data 생성

```

korean_data <- revenue_data %>%
  filter(Artist %in% korean_artists)

```

UI 정의: 사용자 인터페이스

```

ui <- fluidPage(

  # 앱 제목
  titlePanel("Artist Revenue Distribution"),

  # 사이드바 레이아웃: 입력 및 출력 구성
  sidebarLayout(

    # 사이드바 패널: x축에 표시할 특징 선택 및 글로벌 아티스트 수 조절
    # 슬라이더
    sidebarPanel(
      selectInput("xaxis",
                  "Select Feature for X-axis:",
                  choices = names(korean_data)[!(names(korean_data) %in% c("Artist", "Sum_Total_Revenue"))],
                  selected = "Avg_Energy"),
      sliderInput("globalCount",
                  "This number represents the top percentage of global artists:",
                  min = 1.5,
                  max = 100,
                  value = 1.5)
    ),

    # 메인 패널: 결과로서 생성될 분포도 표시
    mainPanel(
      plotlyOutput("distributionPlot")
    )
  )
)

# 서버 로직 정의: 입력에 따라 작동하는 앱의 기능
server <- function(input, output) {

  # x축 고정 범위 계산
  overall_x_limits <- reactive({
    x_values <- revenue_data[[input$xaxis]]
    c(min(x_values, na.rm = TRUE) * 1.1, max(x_values, na.rm

```

```

m = TRUE) * 1.1)
})

# 분포도 생성
output$distributionPlot <- renderPlotly({

  # 선택한 글로벌 아티스트 수에 따른 데이터 필터링
  global_artist_count <- floor((input$globalCount / 100)
* 2000)

  global_data <- revenue_data %>%
    filter(!(Artist %in% korean_artists)) %>%
    arrange(desc(Sum_Total_Revenue)) %>%
    slice(1:global_artist_count)

  # 선택된 데이터 결합 (한국 및 글로벌 아티스트 데이터)
  selected_data <- bind_rows(global_data, korean_data)

  # 한국 및 글로벌 아티스트의 평균 값 계산
  korean_mean <- selected_data %>%
    filter(Artist %in% korean_artists) %>%
    summarise(mean_value = mean(.data[[input$xaxis]], na.
rm = TRUE)) %>%
    pull(mean_value)

  global_mean <- selected_data %>%
    filter(!(Artist %in% korean_artists)) %>%
    summarise(mean_value = mean(.data[[input$xaxis]], na.
rm = TRUE)) %>%
    pull(mean_value)

  # Artist_Type 열 추가 (한국 vs 글로벌 아티스트 구분)
  selected_data <- selected_data %>%
    mutate(Artist_Type = ifelse(Artist %in% korean_artist
s, "Korean Artist", "Global Artist"))

  # 분포도 그리기 (글로벌 아티스트 점 먼저)
  p <- ggplot() +

```

```

    geom_point(data = selected_data %>% filter(Artist_Type == "Global Artist"),
               aes(x = .data[[input$xaxis]], y = Sum_Total_Revenue,
                   color = Artist_Type,
                   text = paste('Artist:', Artist, '<br>Total Revenue:', Sum_Total_Revenue, '<br>', input$xaxis, ':', .data[[input$xaxis]])),
               show.legend = TRUE) +
    geom_point(data = selected_data %>% filter(Artist_Type == "Korean Artist"),
               aes(x = .data[[input$xaxis]], y = Sum_Total_Revenue,
                   color = Artist_Type,
                   text = paste('Artist:', Artist, '<br>Total Revenue:', Sum_Total_Revenue, '<br>', input$xaxis, ':', .data[[input$xaxis]])),
               show.legend = TRUE) +
    # 색상 설정 및 평균선 추가
    scale_color_manual(values = c("Korean Artist" = "red", "Global Artist" = "grey"),
                       labels = c("Korean Artist", "Global Artist")) +
    geom_vline(xintercept = korean_mean, color = "orange", linetype = "dashed", size = 1) +
    geom_vline(xintercept = global_mean, color = "black", linetype = "dashed", size = 1) +
    labs(title = "Distribution of Total Revenue by Selected Feature",
         x = input$xaxis,
         y = "Sum_Total_Revenue") +
    theme_minimal()

# x축 범위 고정
p <- p + xlim(overall_x_limits())

# ggplot을 plotly로 변환
ggplotly(p, tooltip = "text")

```

```

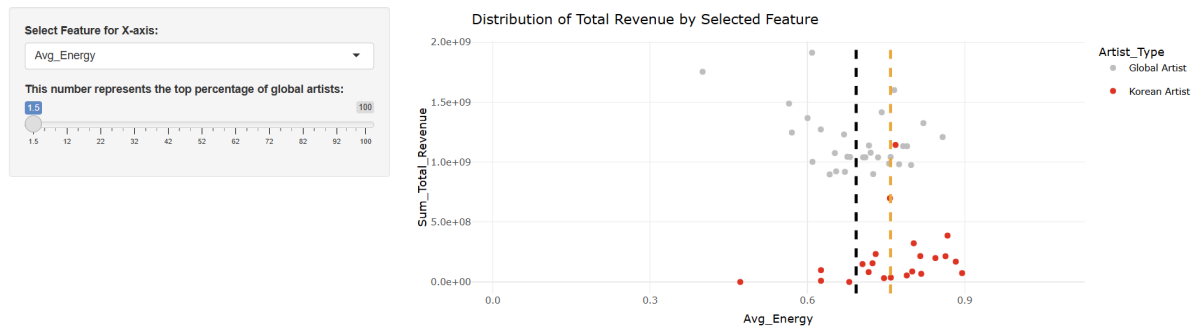
})

}

# Shiny 앱 실행
shinyApp(ui = ui, server = server)

```

Artist Revenue Distribution



- 한국 아티스트는 붉은색 산점도로 글로벌 아티스트는 회색 점으로 표시함.
- x축에 대한 요소의 평균선을 표시하였고 오렌지 세로선이 한국 아티스트 평균선이고, 검은색 세로선이 글로벌 아티스트 평균선이다.
- 왼쪽에는 x축으로 sum of total revenue를 비교할 음악 요소를 선택할 수 있고, 그 아래에는 글로벌 아티스트의 상위 비율을 선택할 수 있다.
 - 예시: 최소 상위1.5% 에서 최대 상위100% 까지 가능 (전체 글로벌 아티스트: 2,000명)

이 shinyApp을 통해 두가지를 제시 할 수 있다.

1. Billboard을 노리는 국내 가수들 (상위 글로벌 아티스트 일부만 참고)
2. Billboard와 Kpop을 노리는 글로벌 가수들 (글로벌 아티스트 전체 참고)

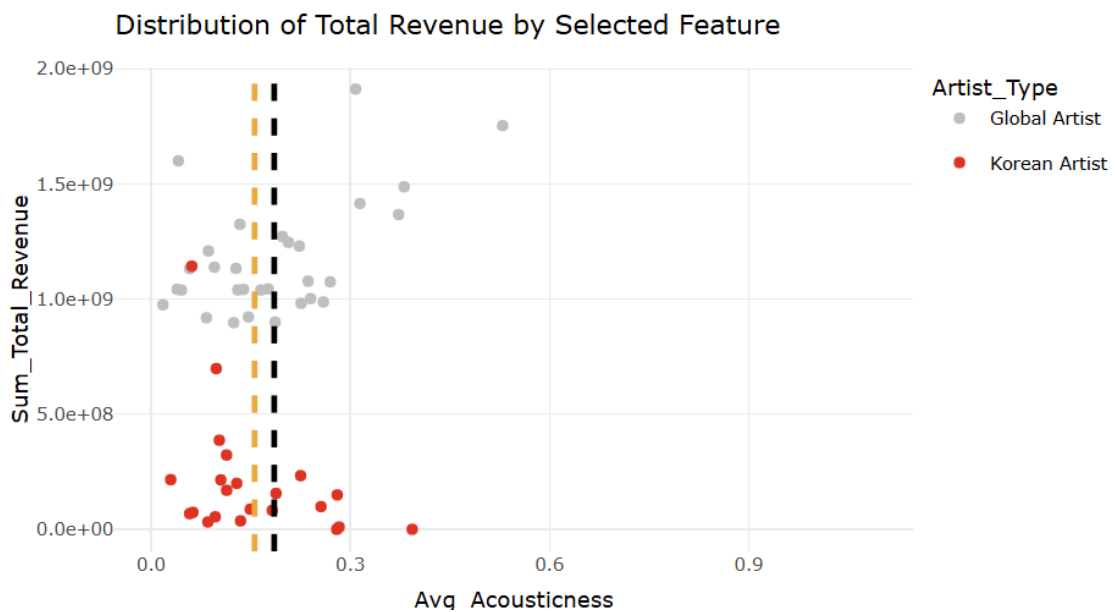
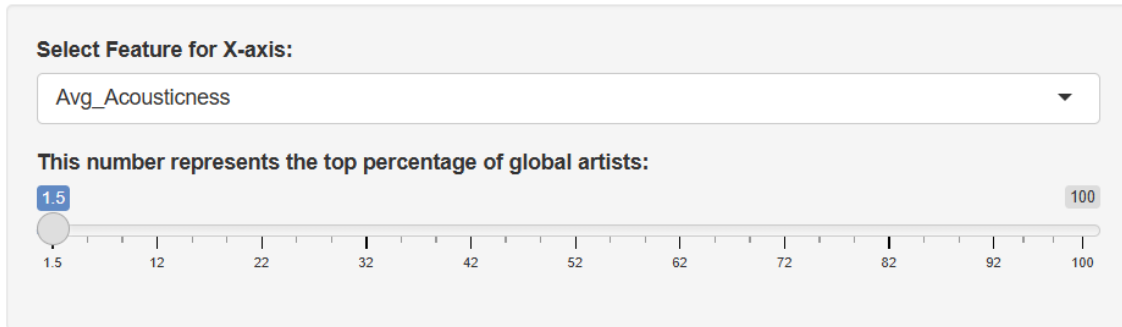
또한 해외 아티스트의 수를 조절하여 그 빈도를 찾고, 각 그룹별 평균 revenue를 계산하여 선으로 그림으로서, 각 변수별 차이를 확인할 수 있음

예시 (Sum of total Revenue by artist VS Average of Acousticness)

아래 사진들은 Sum of total Revenue by artist VS Average of Acousticness를 비교하는 그래프이다.

1. Billboard을 노리는 국내 가수들 (상위 글로벌 아티스트 일부만 참고)

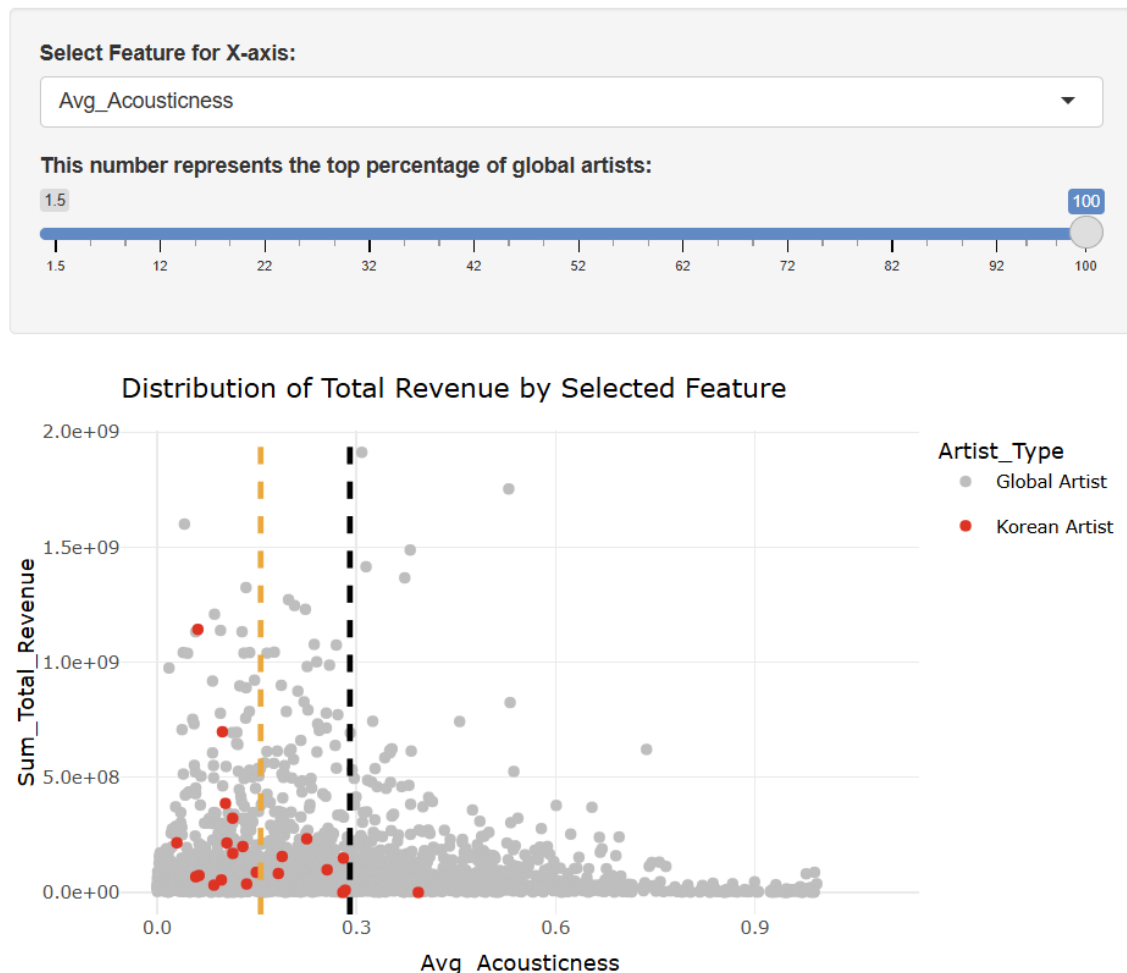
Artist Revenue Distribution



상위 글로벌 아티스트 1.5% (30명) 과 한국 아티스트의 평균을 비교했을 때 글로벌 아티스트들이 조금 더 acousticness가 높은 것을 볼 수 있다. 따라서 Billboard를 목표로 하는 국내 팀들은 acousticness를 조금 더 추가하는 것을 제시한다.

2. Kpop을 노리는 글로벌 가수들 (글로벌 아티스트 전체 참고)

Artist Revenue Distribution



모든 글로벌 아티스트 (2000명)의 평균선을 확인했을 때 상위 1.5%의 글로벌 아티스트들 보다 훨씬acousticness가 높은 것을 볼 수 있다. 따라서 빌보드를 바라보고 있다면 평균적으로 acousticness를 많이 줄이는 것을 제시한다.

모든 글로벌 아티스트 (2000명)의 평균과 한국 아티스트의 평균을 비교했을 때 여전히 해외 아티스트의 acousticness 평균이 더 크다. 하지만 한국의 acousticness 평균은 상위 1.5% 글로벌 아티스트 평균보다 좀 더 낮기 때문에 국내를 타겟하고 있다면 상위 1.5% 글로벌 아티스트의 acousticness평균보다 acousticness를 더욱 많이 줄이는 것을 제시한다.

이와 같은 분석을 통해 음악 요소에 따라 Billboard 및 국내에 관심있는 아티스트에게 음악 요소의 변화를 제시할 수 있다.

Task3: Publish the Shiny app using shinyapps.io and include the URL to the app in the report so that it can be accessed. (If there is another method of

publishing besides shinyapps.io, you may use that method as well.)

Artist Revenue Distribution (shinyapps.io).